

Merge K Sorted Arrays

K=4.

1st Array

n=3

arr=[1,5,9]

2nd Array

n=2

arr=[45,90]

3rd

n=5

arr=[2,6,78,100,234]

4th

n=1

arr=[0]

Output → 0, 1, 2, 5, 6, 9, 45, 78, 90, 100, 234

Solution:-

`ArrayList<int[]> la = new ArrayList<>();`

`la.add(arr1); // [1,5,9]`

`la.add(arr2); // [45,90]`

`la.add(arr3); // [2,6,78,100,234]`

`la.add(arr4); // [0]` → 1, 45, 2, 0
0, 1, 2, 45

`PriorityQueue<ArrayList<Integer>> pq = new PriorityQueue<>();`

`for (int i=0; i< la.size(); i++) {`

```
for (int i = 0; i < la.size(); i++) {
    if (la.get(i).length > 0) {
```

pq $\rightarrow$ {1, i,

Brute Force Approach.

K = 4,

arr1, arr2, arr3, arr4,

```
ArrayList<int[]> ls = new ArrayList<>(),
```

```
for (int i = 0; i < K; i++) {
    int n = sc.nextInt();
    int arr[] = new int[n];
```

```
    for (int j = 0; j < n; j++) {
        arr[j] = sc.nextInt();
    }
```

```
    ls.add(arr);
```

```
}
```

```
ArrayList<Integer> al = new ArrayList<>(),
for (int i = 0; i < ls.size(); i++) {
```

```
for (int i=0; i < ls.size(); i++) {  
    for (int j=0; j < ls.get(i).length; j++) {  
        al.add(ls.get(i)[j]);  
    }  
}
```

```
Collection.sort(al);
```

```
for (int i=0; i < al.size; i++) {  
    s.o.p(al.get(i) + " ");  
}
```

Minimum Cost of ropes 3

$$n = 4$$

$$\text{arr} = [4, 3, 2, 6]$$

1st Way

$$3 + 2 = 5$$

$$\text{cost} = 5$$

$$[4, 5, 6]$$

$$4 + 5 = 9$$

$$\text{cost} = 9$$

$$[9, 6]$$

$$9 + 6 = 15$$

$$\text{Total cost} \rightarrow 5 + 9 + 15 = 29$$

2nd way

$$[4, 3, 2, 6]$$

$$4 + 6 = 10$$

$$\text{cost} = 10$$

$$[10, 3, 2]$$

$$[10, 3, 2]$$

$$10 + 3 = 13$$

$$[13, 2]$$

$$13 + 2 = \underline{15} \quad [15]$$

$$\text{Total cost} = 10 + 13 + 15 = 38$$

1. Create a priority queue
Add all the lengths of ropes

$$\underline{2, 3, 4, 6} \rightarrow \underline{pq}$$

Until we have only value in pq,
we will keep on repeating below
steps.

$$\text{int firstmin} = pq.poll();$$

$$\text{int secondmin} = pq.poll();$$

$$\text{cost} = \text{firstmin} + \text{secondmin}; \quad 1, 2 + 3 = 5$$

$$[4, 6]$$

$$pq.add(\text{cost}); \quad [4, 5, 6]$$

2nd Iteration

1st = 4

2nd = 5

cost = 9

[6, 9]

3rd Iteration

1st = 6

2nd = 9

cost = 15

[15]

Code:-

```
PriorityQueue<Integer> pq = new PriorityQueue();  
for (int i = 0; i < arr.length; i++) {  
    pq.add(arr[i]);  
}
```

```
int total = 0;
```

```
while(pq.size > 1){  
    int firstsmall = pq.poll();  
    int secondsmall = pq.poll();  
    int cost = firstsmall + secondsmall;  
    total += cost;  
    pq.add(cost);  
}
```

Weakest rows

m=5

n=5

k=3

1	1	0	0	0
1	1	1	1	0
1	0	0	0	0
1	1	0	0	0
1	1	1	1	1

```
PriorityQueue<int arr[]> pq =
new PriorityQueue<>((a, b) -> {
    if (a[0] == b[0])
        return a[1] - b[1];
    else
        return a[0] - b[0];
})
```

```
for (int i=0; i<m; i++) {
```

```
    int count=0;
```

```
    for (int j=0; j<n; j++) {
        if (arr[i][j] == 1)
            count++;
    }
```

```
    pq.add(new int[] {count, i});
}
```

```
{ {1, 2}, {2, 0}, {2, 3}, {4, 1}, {5, 4} }
```

```
for (int i=0; i<k; i++) {
```



```
for(int i=0; i<k; i++){  
    s.o.p(pq.poll()[1] + " ");  
}
```